UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Interfaces Java

Readings: ( :chapt. 10)

1

Overview

- Java interfaces
  - An introduction
  - Syntax
  - UML notation
  - Multi-inheritance of interfaces
  - (Some) Java pre-defined interfaces
  - Design pattern: Strategy

2

Recall: Data type

- A data type is a set of values and a collection of operations on this set of values.

  Example: The *int* data type has the following set of values

  { …, -2, -1, 0, 1, 2, … },

  and operations on these values such as: +, *, etc.

Recall Abstract Data Type

- An ADT (abstract data type) is a data type whose values are ***only*** available to clients through the operations in the ADT interface

- In C, a string ***is not*** an ADT. Why?
- and in Java is String an ADT? Why?

Point as an ADT

```
interface IPoint {
    int getX();
    int getY();
    void setX(int x);
    void setY(int y);
    int dist(IPoint p);
}
```


Implementing the IPoint interfa

A class that implements an interface must implement the methods declared in the interface

```
class Point implements IPoint {
    private int x, y;

    public Point (int a, int b) {setX(a); setY(b); }

    public int getX() {return x;}
    public int getY() {return y;}
    public void setX(int a) { (a>=0) ? x=a : System.exit(1);}
    public void setY(int b) { (b>=0) ? x=b : System.exit(1);}

    public int dist(IPoint p) {
        int dx = x – p.getX();
        int dy = y – p.getY();
        return (int) Math.sqrt(dx*dx+dy*dy);
    }
}
```

A client

```
class Point implements IPoint {
    private int x, y;
    public Point (int a, int b) {setX(a); setY(b); }
    public int getX() {return x;}
    public int getY() {return y;}
    public void setX(int
    public void setY(int
    public int dist(Point
        int dx = x – p.g
        int dy = y – p.g
        return (int) Mat
    }
}
```

```
public class Client {
    public static void main(String[] args) {
        IPoint A, B;
        A = new Point(1, 1);
        B = new Point(4, 2);
        System.out.println(A.dist(B));
    }
}
```


Classes and data types

A class is a (possibly partial)
implementation
of a data type.

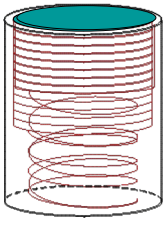
Classes and data types

- Type relates to behaviour while class relates to implementation.
- Two objects from the same class have necessarily the same type.
- Two objects from the same type (*e.g.* IPoint) can be instances from different classes (e.g., CartesianPoint, PolarPoint)

9

Recalling Stacks

- A stack is a last-in-first-out (LIFO) data structure
- Adding an item
  - Referred to as pushing it onto the stack
- Removing an item
  - Referred to as popping it from the stack



10

Recalling Stack

- An ordered collection of data items, that can be accessed at only one end (the top) of the collection
- Basic operations, informally:
  - construct a stack (usually empty)
  - check if it is empty
  - Push: add an element to the top (if it is not full)
  - Top: retrieve the top element (if it is not empty)
  - Pop: remove the top element (if is not empty)

11

Example: interface IStack

```
interface IStack<T> {
int DEFAULT_SIZE=12;
/**
     * PRE: isEmpty() == false
     * POS: this == old this */
    T top();

    /**
     * PRE: isEmpty() == false
     * POS: isFull() == false */
    T pop();

    /**
     * PRE: isFull() == false
     * POS: isEmpty() == false &&
          obj == top()  */
    void push(T obj);
```

By definition Interface variables are static and final because Interfaces do not have instances

12

Example: interface IStack

```
    /**
     * PRE: true
     * POS: return the number of elements in the stack
     */
    int size();

    /**
     * PRE: true
     * POS: return (size() == 0) ? true : false
     */
    boolean isEmpty();

    /**
     * PRE: true
     * POS: return (size() == DEFAULT_SIZE-1) ? true : false
     */
    boolean isFull();
}
```

// return size()==0

Interface basics

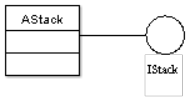
```
class AStack<T> implements IStack<T> {
   //…
}
//…
   IStack<Integer> s;
   s = new IStack<> ();   // ERROR !
   s = new AStack<>();  // OK
```

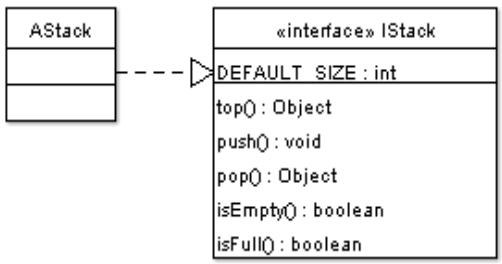
Sintaxe

```
Qualif* interface Ident [extends Ident] [, Ident]*] {
        [Qualif* Type Ident = expression;]*
        [Qualif* Type Ident] ([Type Ident [, Type Ident]*); ]*
```

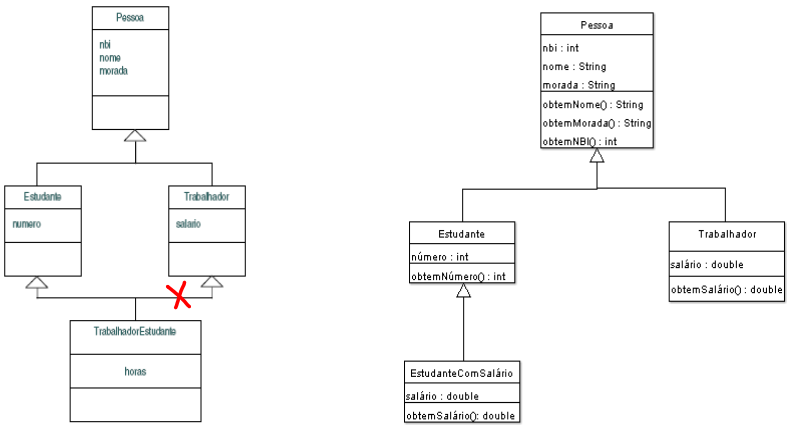
- Qualf qualificador
- Ident Identifier (name)

UML notation



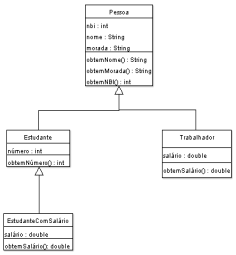


A poor solution for multi-inheritance

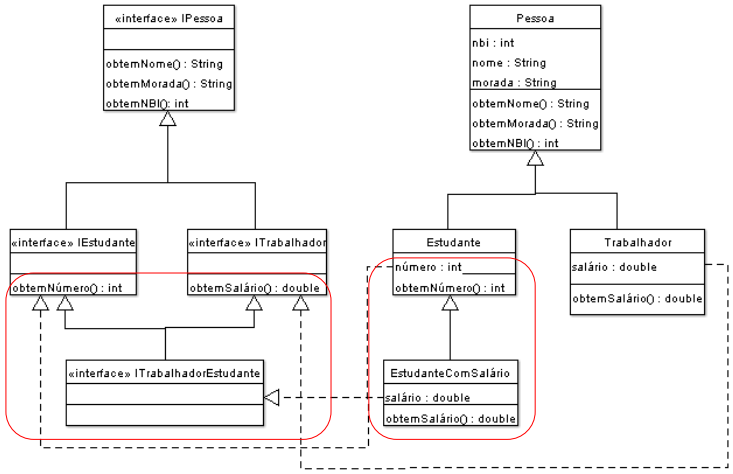


A poor solution for multi-inheritance

```
class Test {
    private static void  f(Trabalhador t) {
        System.out.println (t.obtemSalário());
    }
    void main() {
        EstudanteComSalário ecs = new EstudanteComSalário();
        f(ecs); // ERROR cannot cast to Trabalhador
    }
}
```



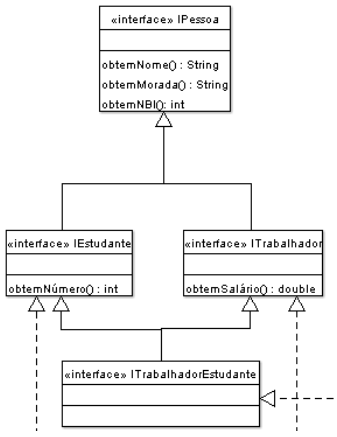
Multi-inheritance of interfaces



19

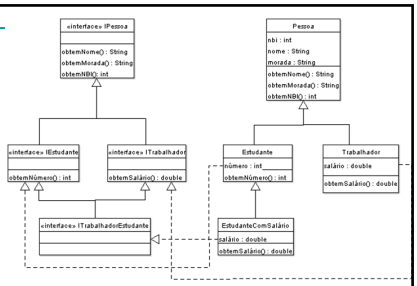
Declaring the interfaces

```
interface IPessoa {
        int nbi();
        String nome();
        String morada();
}
interface IEstudante extends IPessoa {
        int obtemNúmero();
}
interface ITrabalhador extends IPessoa {
        int obtemSalário();
}
interface ITrabalhadorEstudante extends IEstudante, ITrabalhador
{
}
```



20

Implementing the classes

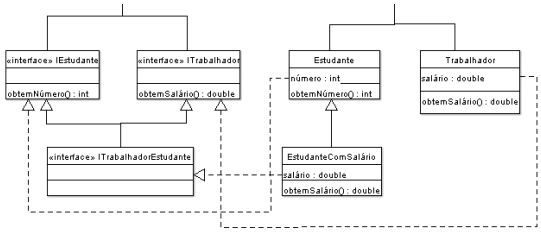



```
class Pessoa {
        // …
}
class Estudante extends Pessoa implements IEstudante {
        //…
}
class Trabalhador extends Pessoa implements ITrabalhador {
        //…
}
class EstudanteComSalário extends Estudante  implements
                                        ITrabalhadorEstudante {
        // …
}
```


21

Required changes in the previous code

```
class Test {
    private static void  f(ITrabalhador t) {
            System.out.println (t.obtemSalário());
    }
    void main() {
            ITrabalhorEstudante ecs = new EstudanteComSalário();
            f(ecs); // OK !!!
    }
}
```



22

Where are we

- Java interfaces
  - Introduction
  - Syntax
  - UML notation
  - Multi-inheritance of interfaces
  - (Some) Java pre-defined interfaces
  - Design pattern: Strategy